

-
1. (10 pts) Assume that the hash table has a size $2^{10} = 1024$, and the universe $U := \{0, 1, 2, \dots, 2^{1000} - 1\}$. Consider a hash family $\mathcal{H} := \{h_b \mid b \in \{0, 1, 2, \dots, 2^{10} - 1\}\}$, where $h_b(x) = x + b \pmod{1024}$. Show that $\mathcal{H}$ is not a universal hash family. (Hint: You can do this by finding two elements/keys $u_1 \neq u_2$ in U such that $\Pr_{h \sim \mathcal{H}}[h(u_1) \neq h(u_2)] > 1/1024$.)

solution

the problem lies with the fact that 1024 is not prime.

consider $u_1 = 0$ and $u_2 = 1024$. when input into h_b , they are guaranteed to result in the same number mod 1024 since $u_1 \pmod{1024} = u_2 \pmod{1024} = 0$.

2. (10 pts) Assume that we have an input sequence of n keys that should be mapped to a table of size n . Bob claims that his hash family is great by saying that for any table slot/bucket, the expected number of keys mapped to the slot is $O(1)$. Briefly explain why this is not a good way to measure the quality of hashing.

solution

the goal of a hash map should be to have the output space be smaller than the universe. furthermore, the process by which elements are mapped is not layed out so collisions of order $O(n)$ are possible.

3. (20 pts) We are given an array of integers $A[1 \dots n]$. We would like to know if there exists i, j, k such that $A[i] = (A[j] + A[k])/2$. Solve this problem in $O(n^2)$ using hashing.

nest two loops, sum over all i, j in n and divide by two. hash each value in the loops. this takes $O(n^2)$ time. then loop over all n and hash each element. if there is a collision, assuming the hashing is collisionless, there exists i, j, k such that $A[i] = (A[j] + A[k])/2$

if we want to find jk , we can store each hash value in a $n \times n$ matrix corresponding to the elements that lead to that value. checking the hashed value of $A[i]$ against the matrix for collisions should reveal the value of jk that leads to the equality. if there is a collision, assuming the hash is universal, there should be on the order of $O(1)$ collisions so checking through each collision should take $O(1)$ time.

4. (30 pts) Consider the following variant of the Interval Selection Problem: We're given n intervals, $I_i = (s_i, f_i)$, where $1 \leq i \leq n$. The length of interval I_i , denoted as $|I_i|$, is defined as $f_i - s_i$. Our goal is to choose a subset of mutually disjoint intervals with the maximum total length. That is, if X is the set of indices of the chosen intervals, their total length will be $\sum_{i \in X} |I_i|$, and we want to maximize this value. We want to show that some natural greedy algorithms fail to find an optimum solution. Assume that all finish times and start times of the intervals are distinct.

- (a) Longest Interval First: In each iteration, choose a longest interval from the remaining intervals (and discard all intervals intersecting the chosen one).

solution

consider $n=3$. the meetings span from $(0, 4), (0, 3), (3, 6)$. the longest interval is the first one, but choosing the first means that no other meeting can be chosen. the optimal is choosing the second and third. the greedy results in $t = 4$, but the optimal is $t = 6$.

- (b) Earliest Finishing Interval First: In each iteration, choose the interval ending the earliest from the remaining intervals.

solution

consider $n=2$. meetings span from $(0, 2), (0, 3)$

greedy results in $t = 2$, picking only the first interval when the optimal is $t = 3$, picking only the second interval.

- (c) Earliest Starting Interval First: In each iteration, choose the interval starting the earliest from the remaining intervals.

solution

consider $n=2$. meetings span from $(0, 2), (1, 4)$

greedy results in $t = 2$, picking only the first interval when the optimal is $t = 3$, picking only the second interval.

5. (30 pts) There are two disjoint groups/sets of items, A and B , i.e., $A \cap B = \emptyset$. Each item i has a value v_i . We want to choose a subset of items with the maximum total value satisfying the following capacity constraints:

- Choose at most u_A items from set A .
- Choose at most u_B items from set B .
- Choose at most u_T items from the union of the two sets, $A \cup B$.

Consider the following greedy algorithm: Consider items in decreasing order of their value. In each iteration, Select the item if it doesn't violate any of the above capacity constraints. To show the optimality of this greedy algorithm, we will take two steps.

- (a) Show that there exists an optimum solution that includes the first item our greedy algorithm selects.

solution

let g be the element selected by the greedy algorithm. if $g \in OPT$, then we are done. suppose $g \notin OPT$, if this is the case, then some other element c was chosen. but necessarily, $v(c) \leq v(g)$. if they are equal, switching c with g results in the same value, or $v(c) < v(g)$ in which case swapping g with c results in a higher value, which is a contradiction.

- (b) Explain why this implies the optimality of the greedy algorithm.

if we run the algorithm $\max(u_t, |A| + |B|)$ times, each time the algorithm runs the problem is converted into the subproblem of

- Choose at most $u_A - 1$ items from set A .
- Choose at most u_B items from set B .
- Choose at most $u_T - 1$ items from the union of the two sets, $A \cup B$.

or

- Choose at most u_A items from set A .
- Choose at most $u_B - 1$ items from set B .
- Choose at most $u_T - 1$ items from the union of the two sets, $A \cup B$.

assuming we update the values of u_A, u_B, u_T at every step.

thus, we can define the original problem recursively until we hit the base case where either there are no more elements in A and B , or $u_T = 0$. since at each step the item picked is optimal, the final solution is as well.